TradingState class

The TradingState class holds all the market information with which your algorithm needs to make decisions with. If you need a more comprehensive overview, you can consult IMC Prosperity's Wiki, we use an adapted version of their TradingState class.

These classes are part of datamodel.py, which is why you do from datamodel import … at the start of your trader Python file. The backtesting environment will have this file, so you don't have to include these classes into your code.

This section is long, but we recommend you read the Order class since it is an easier way to place orders instead of writing everything out.

How to work with these files?

TradingState class Implementation

```
Time = int
Symbol = str
Product = str
Position = int

class TradingState(object):
     def __init__(self,
                  traderData: str,
                  timestamp: Time,
                  listings: Dict[Symbol, Listing],
                  order_depths: Dict[Symbol, OrderDepth],
                  own_trades: Dict[Symbol, List[Trade]],
                  market_trades: Dict[Symbol, List[Trade]],
                  position: Dict[Product, Position],
                  observations: Observation):

          self.traderData = traderData
          self.timestamp = timestamp
          self.listings = listings
```

```
        self.order_depths = order_depths
        self.own_trades = own_trades
        self.market_trades = market_trades
        self.position = position
        self.observations = observations

    def toJSON(self):
        return json.dumps(self, default=lambda o: o.__dict__, sort_keys=True)
```

TradingState is based on a bunch of other classes. The only relevant ones are Trades and OrderDepth. You can ignore Observation.

Trade class

This is basically the implementation for the trades.csv files.

```
Symbol = str
UserId = str

class Trade:
    def __init__(self,
                 symbol: Symbol,
                 price: int,
                 quantity: int,
                 buyer: UserId = None,
                 seller: UserId = None,
                 timestamp: int = 0) -> None:
        self.symbol = symbol
        self.price: int = price
        self.quantity: int = quantity
        self.buyer = buyer
        self.seller = seller
        self.timestamp = timestamp
```

OrderDepth class

The OrderDepth class is an implementation of the prices.csv files.

```
class OrderDepth:
    def __init__(self):
        self.buy_orders: Dict[int, int] = {}
        self.sell_orders: Dict[int, int] = {}
```

Order class

The Order class is the way to send your orders.

```
Symbol = str

class Order:
    def __init__(self, symbol: Symbol, price: int, quantity: int) -> None:
        self.symbol = symbol
        self.price = price
        self.quantity = quantity
```

When you (or Codex) write code, you can create Order objects using Order(symbol, price, qty) and put this into a dictionary with the product name. Appending this object to a dictionary with the symbol and returning it will result in your order being placed.